Application Deployment and Architecture Guide

American Forests Carbon Dashboard

Spatial Informatics Group

October 2026

System Overview

The delivered system consists of a Streamlit carbon dashboard and a FastAPI model service. The dashboard provides site selection, planting-design analysis, carbon estimates, financial analysis, solver workflows, and PDF report generation. The FastAPI service performs model lookup, carbon and protocol calculations, pro forma calculations, scenario evaluation, breakeven solving, registry access, filtered geographic-data delivery, and report generation.

Primary Components

Component	Purpose	Current implementation
Dashboard	User interface for Project Builder, FAQ, and non-production model-management workflows.	Streamlit application; current shared dashboard is deployed at the provided run.app URL.
Model Service API	Executes carbon, finance, scenario, solver, registry, GeoJSON, and report workflows.	FastAPI application in model_service/main.py.
Model / file storage	Stores registered regression models and supporting application data.	Google Cloud Storage (GCS) bucket;
Container images	Versioned deployable application artifacts.	Built from repository Docker configuration and stored in the target cloud container registry.
Source control / CI-CD	Source history, build/test automation, and deployment inputs.	Git repository

User Workflow

- Site Selection: Select or upload a project location and resolve the supported FVS variant/location.
- Planting Design: Define PCT, acreage, survival, site index where applicable, species TPA, protocols, and financial assumptions.
- Carbon Estimates: Calculate modeled carbon and protocol-specific carbon units.
- Project Financials: Calculate annual pro forma rows, net revenue, NPV, and related summaries.
- Solver: Evaluate breakeven acreage, financial sensitivity, and TPA ranges.
- Reporting: Generate a PDF project report from dashboard-provided scenario data.

Repository and Codebase Structure

The following paths are documented in the current API Integration technical reference and should be treated as the starting point for developer orientation.

Path	Responsibility
carbon_dash.py	Streamlit dashboard entry point.
pages/1_project_builder.py	Project Builder: Site Selection, Planting Design, and Solver.
pages/2_faq.py	Frequently Asked Questions page.
pages/3_admin.py	Model Management page; hidden when ENV=production.
model_service/main.py	FastAPI application, endpoints, startup lifecycle, and report generation.
model_service/model.py	Core FVS/model prediction, carbon, pro forma, scenario, and solve logic.
model_service/schemas.py	Pydantic request and response models.
model_service/tpa_sweep.py	TPA breakeven sweep/range solver.
model_service/geo.py	Filtered GeoJSON generation from the model registry.
model_service/config_sync.py	Synchronization of shipped configuration defaults into the model store.
model_service/store/	Model, configuration, and GeoJSON storage abstraction.
aff_dash_client/	Programmatic Python client and CLI for scenario runs.
conf/base/	Shipped configuration: carbon coefficients, pro forma presets, species labels, protocol rules, variant species/presets, and workflow steps.

Functional Implementation Map

The table below connects the dashboard's primary capabilities to the API endpoints, service components, and supporting data responsible for each function. It is intended to provide developers with a high-level view of how the user-facing functionality maps to the backend codebase and can be used as a starting point when troubleshooting, modifying, or extending specific parts of the application.

Functional area	API / service	Implementation / data
Site selection & supported geography	GET /geo/variants; POST /geo/refresh	model_service/geo.py; filtered GeoJSON; model registry
Model availability	GET /models/registry; GET /models/pct-info	Registry + model store; variant/location/PCT metadata
Carbon calculation	POST /carbon/calculate	model_service/model.py; registered FVS-derived models with coefficient fallback
Carbon-unit conversion	POST /carbon/units	model_service/model.py; protocol rules/configuration
Financial calculations	POST /proforma/compute	model_service/model.py; pro forma presets and financial inputs

Scenario evaluation	GET /scenario/defaults; POST /scenario/run; POST /scenario/bulk	model_service/model.py + schemas; registry/model context
TPA solver	POST /scenario/solve-tpa	model_service/tpa_sweep.py
Reporting	POST /reports/generate	model_service/main.py; Quarto runtime and report assets
Model management	Registry/model administration outside production	model_service/store/ + registry/configuration; pages/3_admin.py

Runtime Architecture

At runtime, the dashboard collects user inputs and sends HTTPS requests to the FastAPI model service. The API resolves the applicable model context, reads registered model/supporting files, performs calculations or solver operations, and returns structured results to the dashboard. Report requests additionally invoke the Quarto report-generation workflow.

Current Cloud Run implementation:

The Streamlit dashboard is served on port 8080. The FastAPI service runs ‘uvicorn model_service.main:app on container port 8080’. The dashboard resolves the model-service URL from Streamlit secrets, CARBON_API_BASE_URL, or the local default http://127.0.0.1:8001. Production configuration rejects localhost API URLs.

1. The user opens the Streamlit dashboard and selects a supported project location.
2. Dashboard resolves variant/location and retrieves registry-aware options from the API.
3. User supplies planting, protocol, financial, or solver inputs.
4. Dashboard submits the corresponding request to the FastAPI service.
5. API resolves variant, location, PCT, species, and model context; registered FVS-derived models are used when available, with coefficient-based carbon calculations as the documented fallback.
6. API returns carbon, financial, scenario, solver, GeoJSON, or report output to the dashboard.

Model Service Startup Behavior

On startup, the FastAPI lifespan process loads the model registry, synchronizes default configuration from conf/base, and builds the cached filtered GeoJSON used by the dashboard map. Model files are loaded lazily on first use rather than loading all model files at startup.

Environment Separation

Environment	Dashboard	API	Purpose / notes
Development	LINK	LINK	Developer testing; Model Management may be visible when ENV is not production.
Production	LINK	LINK	AF-facing deployment; ENV=production hides Model Management.

External Services and Storage

Verified deployment resources:

- Region: us-west1
- Model buckets: ‘af-carbon-models-dev’ and, ‘af-carbon-models-prod’
- Upload buckets: ‘af-carbon-upload-dev’ and ‘af-carbon-upload-prod’

The model bucket stores registry.json, models/, configuration, and cached GeoJSON. The upload bucket stores uploaded geometry features when uploaded from site selection and removes the features after a certain period of time.

Dependency	What it provides / contains
Cloud Run - Dashboard	Hosts the Streamlit dashboard application.
Cloud Run - Model Service API	Hosts FastAPI carbon/model/solver/report service.
Container registry	Stores built dashboard/API container images.
GCS Bucket	Registered .pkl models, registry/configuration, filtered geographic/supporting files as configured.
Quarto	Renders project PDF reports.
Git repository	Application source, configuration, Docker/deployment files, tests.

Asset Inventory

Asset	Expected contents	Final delivery
Regression model package	Complete delivered .pkl model set organized by Variant Location and PCT level.	Cloud bucket: models/*.pkl
Model registry	registry.json or equivalent inventory controlling available variant/location/PCT combinations.	Cloud bucket: registry.json
Variant/location geography	Filtered/supporting GeoJSON used for supported-location selection.	Cloud bucket: geo/*.geojson
Base configuration	Carbon coefficients, pro forma presets, species labels, protocol rules, variant presets/species, workflow steps.	Repository: conf/base/ Cloud bucket: config/
Report assets	Quarto templates and images/assets required for PDF generation.	Repository: model_service/quarto/
UI assets	Images/static files required by dashboard.	Repository: - pages/ - utils/functions/

Required Dependencies and Local Environment Setup

This section is also detailed in the README in the GitHub repository.

Configuration Reference

Setting	Purpose	Requirement
CARBON_API_BASE_URL	Resolves the deployed model-service base URL used by the dashboard.	Set to the API URL in deployed dashboard environments.
ENV	Controls environment-specific behavior, including visibility of Model Management.	Set to production for the production dashboard.
Model-store configuration	Controls where models, registry/config, and supporting files are read from.	Record exact production values/path and access method.
Cloud project / region	Identifies target cloud resources.	Record AF production project ID and region.
Runtime service account	Identity used by the deployed service.	Record service account email and granted roles.
Report runtime / assets	Required by /reports/generate.	Quarto and report assets must be present in the runtime image.

Local Environment Setup

The application uses 'uvicorn' (uv) for Python dependency and environment management. The repository's authoritative dependency specification consists of pyproject.toml and uv.lock. pyproject.toml defines the project and its required dependencies, while uv.lock records the resolved dependency versions used to create a reproducible environment. These files should remain under version control and should be used when configuring development environments or building deployment images.

Developers should use uv rather than independently installing packages with pip or maintaining a separate requirements.txt. This helps ensure that local development environments remain consistent with the dependency definitions maintained in the repository.

After cloning the repository, uv must be installed on the development machine. Installation instructions differ by operating system; the repository README provides the installation command for macOS/Linux and directs Windows users to the official uv installation documentation.

The recommended workflow does not require manually activating a Python virtual environment. uv run will create the repository's .venv if necessary, synchronize the required dependencies, and execute the requested command within that environment.

For developers who prefer to work inside an activated virtual environment, the environment can instead be initialized manually with:

```shell
uv sync
source .venv/bin/activate
```

Running uv sync creates the .venv and synchronizes its dependencies from pyproject.toml and uv.lock.

Required Data Preparation

Before running the application for the first time, the repository's data-preparation step should be executed from the repository root:

```
Shell
make
```

The Makefile downloads the FVS Variants shapefile and converts it to a simplified GeoJSON used by the application. During this process, the available variants are filtered using the supported FVS variant definitions maintained in:

```
Shell
conf/base/supported_variants.txt
```

This step prepares the geographic data required by the application's site-selection and FVS variant workflows and should be completed when configuring a clean development environment.

Running the Dashboard Locally

The recommended method for starting the Streamlit dashboard is:

```
Shell
uv run streamlit run carbon_dash.py
```

Using uv run avoids the need to manually activate .venv. If necessary, uv creates the environment, synchronizes the project dependencies, and then starts the Streamlit application.

If the environment has already been manually activated, the equivalent command is:

```
Shell
streamlit run carbon_dash.py
```

The uv run approach is preferred for developer setup because it provides a consistent entry point from a clean repository checkout and reduces the number of environment-management steps required before starting the application.

Running the FastAPI Service Locally

The FastAPI model service can also be run locally when development or testing requires the dashboard and API to operate entirely within the local environment. The API should be started in a separate terminal with:

```
Shell
uv run uvicorn model_service.main:app --reload --port 8001
```

This starts the FastAPI service at:

```
Shell
http://127.0.0.1:8001
```

When CARBON_API_BASE_URL is not explicitly set, the locally running dashboard uses this address by default. This allows developers to modify and test the dashboard and model service together without deploying API changes to Cloud Run. The --reload option also causes the local FastAPI development server to reload as source code changes are made.

A typical fully local development session therefore consists of two processes running in separate terminals:

```
Shell
Terminal 1 – Model service
uv run uvicorn model_service.main:app --reload --port 8001


Terminal 2 – Dashboard
uv run streamlit run carbon_dash.py
```

In this configuration, CARBON_API_BASE_URL should remain unset so that the dashboard uses the local API.

Running the Local Dashboard Against the Cloud Run FastAPI

The dashboard can also be run locally while using a deployed Cloud Run instance of the FastAPI model service. This is useful when dashboard changes need to be tested against an existing development or production API without running the model service locally.

In this configuration, CARBON_API_BASE_URL must be set to the appropriate Cloud Run service URL before starting Streamlit.

On Linux:

```
None
export CARBON_API_BASE_URL="https://YOUR-CLOUD-RUN-URL"
uv run streamlit run carbon_dash.py
```

On Windows PowerShell:

```
Shell
$env:CARBON_API_BASE_URL = "https://YOUR-CLOUD-RUN-URL"
uv run streamlit run carbon_dash.py
```

CARBON_API_BASE_URL therefore determines whether the locally running dashboard communicates with the default local FastAPI service or a hosted API. When the variable is unset, the application defaults to the local service at http://127.0.0.1:8001; when it is set, requests are directed to the specified hosted service.

Cloud Run Build and Deployment Procedure

The AF Dash application is deployed to Google Cloud Run as two separate containerized services: a FastAPI model service and a Streamlit dashboard service. Both services are built from the same repository and Python dependency manifest but use separate Dockerfiles. The API is built from Dockerfile, while the dashboard is built from Dockerfile.dashboard.

The repository contains an automated deployment workflow at .github/workflows/cd.yml. The workflow runs when changes are pushed to either dev or main. The dev branch deploys development resources, while main deploys production resources. The API is deployed first because the resulting Cloud Run API URL is passed to the dashboard as CARBON_API_BASE_URL.

This procedure distinguishes between two deployment paths: an initial manual deployment used to establish and validate a new Google Cloud environment, and the repository's ongoing GitHub Actions CI/CD deployment used after Google Cloud, IAM, Workload Identity Federation (WIF), and GitHub repository secrets have been configured.

Deployment Architecture and Methods

Both deployment paths produce the same application architecture. The difference is how the deployment is initiated and authenticated. The initial procedure uses an authenticated developer's local Google Cloud CLI session. Ongoing deployments are initiated by pushes to dev or main and authenticate from GitHub Actions to Google Cloud using WIF.

Initial Google Cloud Environment Setup

This section applies when establishing a new AF-controlled environment before GitHub CI/CD is available. These steps are generally one-time provisioning tasks rather than part of every application release.

Authenticate from the workstation that will perform the initial deployment:

```
Shell
gcloud auth login
gcloud auth application-default login

$PROJECT_ID="YOUR_AF_PROJECT_ID"
$REGION="us-west1"

gcloud config set project $PROJECT_ID
```

Enable the Google Cloud services required by the current build and runtime architecture:

```
Shell
gcloud services enable `
  run.googleapis.com `
  cloudbuild.googleapis.com `
  storage.googleapis.com `
  iamcredentials.googleapis.com `
  serviceusage.googleapis.com `
  --project=$PROJECT_ID
```

Before proceeding, confirm that the project can publish the gcr.io container image names used by the current repository workflow. If AF changes the image registry architecture during handoff, the manual commands and cd.yml should be updated together.

Initial Runtime Resources and IAM

Before the initial manual deployment, define the production resources and create the runtime service account used by the Cloud Run services. The values below follow the current production naming convention in cd.yml and should be confirmed for the final AF environment.

```
Shell
$API_SERVICE="model-service-api-prod"
$DASH_SERVICE="af-dash-prod"

$API_IMAGE="gcr.io/$PROJECT_ID/model-service-api-prod:initial"
$DASH_IMAGE="gcr.io/$PROJECT_ID/af-dash-prod:initial"

$MODEL_BUCKET="af-carbon-models-prod"
$UPLOAD_BUCKET="af-carbon-upload-prod"

$RUNTIME_SA_NAME="af-dash-runtime"
$RUNTIME_SA="$RUNTIME_SA_NAME@$PROJECT_ID.iam.gserviceaccount.com"
```

Create the Runtime Service Account

Create a dedicated service account for the Cloud Run services:

```
Shell
gcloud iam service-accounts create $RUNTIME_SA_NAME `
  --project=$PROJECT_ID `
  --display-name="AF Dash Cloud Run Runtime"
```

The runtime service account is used by the deployed application to access the Google Cloud resources it requires. Permissions should be limited to those needed by the application at runtime.

Create the Cloud Storage Buckets

Create or verify the model and upload buckets. The current CI/CD workflow automatically creates the model bucket if it is absent; for an initial manual deployment, the resources should be explicitly verified before the containers are deployed.

Create the buckets if they do not already exist:

```
Shell
gcloud storage buckets create "gs://$MODEL_BUCKET" `
  --project=$PROJECT_ID `
  --location=$REGION `
  --uniform-bucket-level-access

gcloud storage buckets create "gs://$UPLOAD_BUCKET" `
  --project=$PROJECT_ID `
  --location=$REGION `
  --uniform-bucket-level-access
```

If a bucket already exists, do not recreate it. Confirm instead that the expected production models, registry/configuration, geographic data, and supporting assets are present and accessible to the runtime identity.

Grant Bucket Access to Service Account

Grant the runtime service account access to the production model store:

```
Shell
gcloud storage buckets add-iam-policy-binding "gs://$MODEL_BUCKET" `
  --member="serviceAccount:$RUNTIME_SA" `
  --role="roles/storage.objectUser"
```

Grant the runtime service account access to the production upload store:

```
Shell
gcloud storage buckets add-iam-policy-binding "gs://$UPLOAD_BUCKET" `
  --member="serviceAccount:$RUNTIME_SA" `
  --role="roles/storage.objectUser"
```

Grant Signed URL Permissions

The application requires the runtime service account to sign URLs for Cloud Storage objects. Grant the service account permission to generate credentials on its own behalf:

```
Shell
gcloud iam service-accounts add-iam-policy-binding $RUNTIME_SA `
  --project=$PROJECT_ID `
  --member="serviceAccount:$RUNTIME_SA" `
  --role="roles/iam.serviceAccountTokenCreator"
```

After these steps, the runtime service account should have access to both application buckets and the permissions required for signed URL generation. The service account is then supplied to the Cloud Run services using the --service-account option during deployment.

Initial Data Preparation

The deployed application requires the FVS variants GeoJSON in the model store. The automated workflow checks for geo/FVS_Variants_and_Locations_4326_simplified_full.geojson and creates it from the USDA FVS Variants shapefile when it is missing. The initial environment must contain the same asset before application validation.

The production model bucket must also contain the production model registry, models, configuration, and any other supporting assets required by the model service. Creating an empty bucket is not sufficient for a production deployment.

Initial Manual Build and Deployment

The first deployment can be completed entirely from an authenticated local Google Cloud CLI session. GitHub Actions, WIF, and GitHub repository secrets are not required for this step.

Build the FastAPI Image

The API is built from the repository root using Dockerfile. Python dependencies are resolved from pyproject.toml and uv.lock, and the API image includes the report-generation runtime required by the service.

```
Shell
gcloud builds submit . `
  --project=$PROJECT_ID `
  --tag=$API_IMAGE
```

Deploy the FastAPI Service

```
Shell
gcloud run deploy $API_SERVICE `
  --image=$API_IMAGE `
  --project=$PROJECT_ID `
  --region=$REGION `
  --platform=managed `
  --allow-unauthenticated `
  --service-account=$RUNTIME_SA `
  --timeout=3600 `
  --cpu=4 `
  --memory=4Gi `

--set-env-vars="GCP_PROJECT=PROJECTID,MODELSTOREBACKEND=gcs,MODELSTOREBUCKET=MODEL_BUC
KET,BULK_WORKERS=4,BULK_WORKER_MODE=process"
```

Capture and Validate the API URL

```
Shell
$API_URL = gcloud run services describe $API_SERVICE `
  --project=$PROJECT_ID `
  --region=$REGION `
  --format="value(status.url)"

curl "$API_URL/health"
```

Also open $API_URL/docs and confirm that the expected FastAPI endpoints are available before deploying the dashboard.

Build the Streamlit Dashboard Image

The dashboard uses Dockerfile.dashboard. Because the repository root also contains the API Dockerfile, the dashboard build must explicitly select Dockerfile.dashboard. The current CI/CD workflow does this with a temporary Cloud Build configuration. The same pattern can be used manually:

```
Shell
@"
steps:
  - name: gcr.io/cloud-builders/docker
    args: ['build', '-f', 'Dockerfile.dashboard', '-t', '$DASH_IMAGE', '.']
```

```
images: ['$DASH_IMAGE']
"@ | Set-Content cloudbuild-dashboard.yaml

gcloud builds submit . `
  --project=$PROJECT_ID `
  --config=cloudbuild-dashboard.yaml
```

Deploy the Streamlit Dashboard

```
Shell
gcloud run deploy $DASH_SERVICE `
  --image=$DASH_IMAGE `
  --project=$PROJECT_ID `
  --region=$REGION `
  --platform=managed `
  --allow-unauthenticated `
  --service-account=$RUNTIME_SA `

--set-env-vars="CARBON_API_BASE_URL=APIURL,ENV=production,GCPPROJECT=PROJECT_ID,MODEL_STORE_
BACKEND=gcs,MODEL_STORE_BUCKET=MODELBUCKET,GOOGLESERVICEACCOUNTEMAIL=RUNTIME_SA,SIT
E_UPLOAD_BUCKET=$UPLOAD_BUCKET"
```

CI/CD Prerequisites and GitHub Deployment Configuration

After the manual environment has been validated, configure GitHub Actions to perform ongoing deployments. The current workflow authenticates to Google Cloud using Workload Identity Federation rather than a long-lived service-account JSON key.

GitHub secret	Purpose
GCP_PROJECT_ID	Google Cloud project containing the AF Dash resources.
GCP_WIF_PROVIDER	Workload Identity Federation provider that establishes the GitHub-to-Google Cloud trust relationship.
GCP_SERVICE_ACCOUNT	Google Cloud service account assumed by GitHub Actions. The current workflow also assigns this identity to both Cloud Run services at runtime.

The WIF provider, IAM trust relationship, service-account permissions, and these repository secrets must exist before cd.yml can deploy successfully. The project ID and service-account email are configuration identifiers; no long-lived service-account private key is stored by the current workflow.

Development and Production CI/CD Configuration

The workflow runs on pushes to dev and main and derives the resource names from the branch.

Resource	Development (dev)	Production (main)
API service	model-service-api-dev	model-service-api-prod
Dashboard service	af-dash-dev	af-dash-prod
API image	gcr.io/<PROJECT>/model-service-api-dev:<SHA>	gcr.io/<PROJECT>/model-service-api-prod:<SHA>

Dashboard image	gcr.io/<PROJECT>/af-dash:<SHA>	gcr.io/<PROJECT>/af-dash-prod:<SHA>
Model bucket	af-carbon-models-dev	af-carbon-models-prod
Upload bucket	af-carbon-upload-dev	af-carbon-upload-prod
ENV	development	production
Region	us-west1	us-west1

Images are tagged with the Git commit SHA, allowing deployed revisions to be associated with the source revision that produced them.

Automated GitHub Actions Deployment

Once WIF and the GitHub secrets are configured, normal development and production releases should use .github/workflows/cd.yml. The workflow performs the following sequence:

1. Check out the source revision.
2. Determine development or production configuration from the source branch.
3. Create environment-specific service, image, and bucket names.
4. Authenticate to Google Cloud using WIF and GCP_SERVICE_ACCOUNT.
5. Configure the Google Cloud SDK.
6. Create the model-store bucket if it does not already exist.
7. Check for and, if necessary, generate and upload the FVS GeoJSON.
8. Build the FastAPI image from Dockerfile.
9. Build the Streamlit image from Dockerfile.dashboard.
10. Deploy the API with the environment-specific model-store configuration.
11. Retrieve the deployed API Cloud Run URL.
12. Deploy the dashboard with the API URL assigned to CARBON_API_BASE_URL.

The current API deployment uses a 3,600-second timeout, 4 CPU, 4 GiB memory, BULK_WORKERS=4, BULK_WORKER_MODE=process, and unauthenticated access. Both services currently use the service account supplied through GCP_SERVICE_ACCOUNT as their Cloud Run runtime identity.

Deployment Validation Procedure

A successful container deployment is not sufficient by itself. Run these checks after a new deployment or major configuration change. Record the deployed release/commit and the result of each test.

#	Validation	Expected result
1	API health: open/call GET /health.	HTTP 200 and healthy service response.
2	API documentation: open the API /docs endpoint.	Swagger/OpenAPI interface loads.
3	Registry: call GET /models/registry.	Registered model inventory is returned.
4	Supported geography: call GET /geo/variants.	Valid filtered GeoJSON is returned.
5	Dashboard/API connection: open production dashboard.	Dashboard loads and no model-service connection error is shown.
6	Site Selection: use a known supported location.	Variant/location resolves and downstream workflow becomes available.
7	Carbon calculation: use a known non-zero species TPA scenario.	Carbon results/charts populate and model_source is returned as documented.
8	Protocol/carbon units: select at least one supported protocol.	Carbon-unit results populate.
9	Financial calculation: run Project Financials.	Pro forma rows and summary metrics such as NPV populate.
10	Solver: run a known breakeven acreage or TPA scenario.	Solver returns a feasible result/interval when the test case is expected to be feasible.
11	Report generation: generate a project report.	PDF is produced successfully.
12	Production-mode behavior.	Model Management is hidden when ENV=production.

Known Good Example Scenario

Field	Acceptance value
Variant/location	East Cascades (EC-613) - Snoqualmie National Forest
PCT level	None
Species TPA	PP: 60, DF: 15, WL: 20, WP: 20, Total: 115
Survival percentage	70
Site index	80
Net acres	10,000
Protocol	GS
Financial assumptions	Default
Expected carbon checkpoint	$-8,921,700

Expected NPV (Year 40) checkpoint	$-8,921,700
Total Net Revenue	$104,254,300

Operational Notes and Troubleshooting

Additional operational notes:

- Registry.json controls available variant/location/PCT combinations
- Filtered GeoJSON exposes only locations with registered model coverage
- Model files are expected under models/ in the configured model store
- Quarto is installed in the API image and its version is checked during the Docker build
- Bulk scenarios default to MAX_BULK_BATCH_SIZE=1000 and may use process or thread workers according to BULK_WORKER_MODE

Symptom	Checks
Dashboard loads but outputs are empty	Confirm a supported location is selected; species TPA is not all zero; protocols are selected; API base URL is correct; registry entries exist for selected variant/location/PCT.
Location/model options unavailable	Check registry contents, filtered GeoJSON generation/cache, selected variant/location, and model-store access.
Carbon endpoint returns fallback output	Confirm matching registered FVS model exists for variant/location/PCT. The documented service can fall back to coefficient-based carbon calculations when FVS models are unavailable.
Report generation fails	Confirm Quarto and report assets are present in the API runtime and writable temporary/output paths are available.
Model Management appears in production	Confirm ENV=production.
New model not visible	Confirm file structure/name is valid, registry/config metadata is updated, and the registry/GeoJSON cache is refreshed as required.